

Deadlock Modelling, Prevention and Avoidance

FOSD – Unit 4: Deadlock Modeling and System-level Handling

Prashant Jadav

Department of Computer Engineering
DEPSTAR – CHARUSAT

Learning Objectives

- Define deadlock and explain it with practical examples.
- Distinguish reusable and consumable resources.
- Model dependencies using a Resource Allocation Graph (RAG).
- Explain the four necessary conditions for deadlock.
- Explain and compare deadlock prevention and avoidance.
- Apply Banker's Algorithm to find a safe sequence.
- Evaluate a resource request before granting it.

What is Deadlock?

Definition

A deadlock is a state in which a set of processes is permanently blocked because every process is waiting for a resource or event that can only be supplied by another process in the same set.

- Processes cannot make further progress.
- Processes may hold resources while waiting for others.
- The waiting dependency can form a cycle.

Simple example

P1 holds a printer and waits for a scanner.

P2 holds the scanner and waits for the printer.

Neither process can continue.

Real-World Analogy: Traffic Gridlock

Traffic analogy

- Each car represents a process.
- Each road segment represents a resource.
- A car holds its current segment.
- It waits for the next segment.
- If every car waits for another car, no car moves.

OS interpretation

Processes $\rightarrow$ Resources

Resources $\rightarrow$ Processes

The same cyclic dependency can occur with devices, locks, files, buffers, or other resources.

Classic Deadlock Example

Current state

- P1 holds R1.
- P2 holds R2.
- P1 requests R2.
- P2 requests R1.

Waiting cycle

$$P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$$

P1 waits for P2.

P2 waits for P1.

Result

No process in the cycle can release its held resource, so the processes remain blocked.

Resource Model

A system contains:

- A finite set of resource types $R_1, R_2, \dots, R_m$.
- One or more instances of each resource type.
- Processes that request, use, and release resources.

Typical resources

CPU cores, memory frames, printers, disks, device controllers, mutexes, semaphores, file locks and database locks.

Request $\rightarrow$ Use $\rightarrow$ Release

Reusable Resources

Definition

A reusable resource can be allocated, used, released, and allocated again.

- CPU
- Memory frames
- Printer
- Disk
- Mutex / lock
- File lock

Lifecycle

Free → Allocated → Used → Released → Free

Consumable Resources

Definition

A consumable resource is produced and then consumed; the consumed instance is no longer available.

- Messages
- Signals/events
- Data items in a buffer
- Event notifications

Example

Producer → Message Queue → Consumer

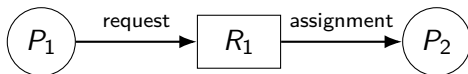
A consumer may wait for a message that another process must produce.

Reusable vs. Consumable Resources

Property	Reusable	Consumable
Nature	Returned after use	Consumed during use
Examples	CPU, printer, lock, memory	Message, signal, event
Lifecycle	Allocate/use/release	Produce/consume
Availability	Can become free again	New instances must be produced
Deadlock relevance	Very common	Possible through waiting dependencies

Resource Allocation Graph (RAG)

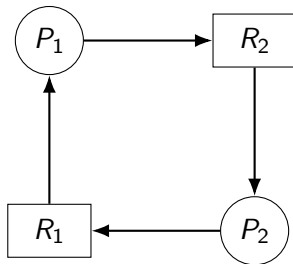
- Process node: circle, e.g. P_1 .
- Resource node: rectangle, e.g. R_1 .
- Request edge: $P_i \rightarrow R_j$.
- Assignment edge: $R_j \rightarrow P_i$.



P_1 waits for R_1 , while R_1 is currently allocated to P_2 .

RAG: Deadlock Cycle

$P_1 \rightarrow R_2, \quad R_2 \rightarrow P_2, \quad P_2 \rightarrow R_1, \quad R_1 \rightarrow P_1$



Single-instance resources

A cycle in the RAG is a necessary and sufficient condition for deadlock.

Cycle vs. Deadlock

Single instance of each resource type

Cycle in RAG $\iff$ Deadlock

A cycle is necessary and sufficient.

Multiple instances

Cycle in RAG $\Rightarrow$ Deadlock may exist

A cycle is necessary, but not sufficient.

Reason

Another instance of the resource may satisfy a waiting request, allowing a process to finish and release resources.

Four Necessary Conditions for Deadlock

All four must hold simultaneously:

- ① **Mutual Exclusion**: at least one resource is non-shareable.
- ② **Hold and Wait**: a process holds resources while waiting for more.
- ③ **No Preemption**: an allocated resource cannot be forcibly taken.
- ④ **Circular Wait**: a circular chain of processes exists.

Key principle

If the OS permanently prevents any one condition, deadlock cannot occur.

Condition 1: Mutual Exclusion

Meaning

At least one resource is non-shareable and only one process can use it at a time.

Example

A printer cannot safely process two independent print streams through the same physical output path simultaneously.

Prevention idea

Make resources shareable where technically possible, or use techniques such as spooling for devices such as printers.

Condition 2: Hold and Wait

Meaning

A process holds one or more resources while waiting for additional resources.

$$P_1 : \underbrace{R_1}_{\text{held}} + \underbrace{R_2}_{\text{waiting}}$$

Prevention

- 1 Require the process to request all needed resources at once.
- 2 Require it to release held resources before requesting new ones.

Trade-off: this can reduce concurrency and resource utilization.

Condition 3: No Preemption

Meaning

A resource cannot be forcibly removed from a process that currently holds it.

Prevention idea

If a requested resource is unavailable, the process may be forced to release suitable held resources and retry later.

Practical limitation

CPU time is naturally preemptible, while a partially completed printer operation may not be safely preemptible.

Condition 4: Circular Wait

Meaning

A cycle exists:

$$P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n \rightarrow P_1$$

where each process waits for a resource held by the next process.

Prevention: Resource Ordering

Assign a total order:

$$R_1 < R_2 < R_3 < \dots < R_m$$

and require processes to request resources only in increasing order.

Example

If P already holds R_3 , it cannot request R_1 later.

Deadlock Prevention: Summary

Condition	Prevention approach	Trade-off
Mutual exclusion	Share resources / spooling where possible	Limited applicability
Hold and wait	Request all or release before new request	Lower concurrency
No preemption	Preempt/release when possible	Not universal
Circular wait	Impose global resource ordering	Restricts request order

Core idea

Prevention is conservative: deliberately restrict resource usage so at least one deadlock condition never holds.

Deadlock Avoidance

Idea

Deadlock avoidance makes allocation decisions dynamically so that the system does not enter an unsafe state.

- The OS evaluates a request before granting it.
- The request is granted only if the resulting state remains safe.
- The process's maximum possible resource requirements are generally known.



Safe State vs. Unsafe State

Safe state

There exists at least one order in which every process can obtain its remaining resources, finish, and release its allocated resources.

Unsafe state

The OS cannot guarantee completion of all processes under their declared maximum requirements.

Important

Unsafe state $\neq$ Deadlock necessarily

An unsafe state means deadlock may become possible.

Banker's Algorithm

Banker's Algorithm is a deadlock-avoidance algorithm for systems with multiple instances of resource types.

Data structures

- **Available:** currently available instances.
- **Max:** maximum demand of each process.
- **Allocation:** currently allocated resources.
- **Need:** remaining maximum demand.

$$\text{Need} = \text{Max} - \text{Allocation}$$

The algorithm searches for a safe sequence.

Banker's Algorithm: Safety Algorithm

Initially:

$$Work = Available, \quad Finish[i] = false$$

Repeat:

- 1 Find P_i such that

$$Finish[i] = false \quad \text{and} \quad Need_i \leq Work.$$

- 2 Assume P_i finishes.

- 3 Release its allocation:

$$Work = Work + Allocation_i.$$

- 4 Set $Finish[i] = true$.

If all processes can be marked finished, the state is safe.

Banker's Algorithm: Worked Example

Assume resources A, B and C.

	Allocation			Max		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

Available

$$Available = (3, 3, 2)$$

$$Need = Max - Allocation$$

Worked Example: Need Matrix

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Initial:

$$Work = (3, 3, 2)$$

- P1: $(1, 2, 2) \leq (3, 3, 2) \Rightarrow$ can finish.
- P3: $(0, 1, 1) \leq (3, 3, 2) \Rightarrow$ can finish.
- P0, P2 and P4 initially need more A than available.

Worked Example: Constructing a Safe Sequence

Start with $Work = (3, 3, 2)$. Choose a process whose Need is no greater than $Work$.

Step	Process completed	New Work
1	P_1	$(3, 3, 2) + (2, 0, 0) = (5, 3, 2)$
2	P_3	$(5, 3, 2) + (2, 1, 1) = (7, 4, 3)$
3	P_4	$(7, 4, 3) + (0, 0, 2) = (7, 4, 5)$
4	P_0	$(7, 4, 5) + (0, 1, 0) = (7, 5, 5)$
5	P_2	$(7, 5, 5) + (3, 0, 2) = (10, 5, 7)$

Safe sequence

$\langle P_1, P_3, P_4, P_0, P_2 \rangle$

All processes can finish, so the initial state is safe.

Banker's Algorithm: Resource Request

For a request $Request_i$:

- 1 Check:

$$Request_i \leq Need_i?$$

If no, the process exceeds its declared maximum.

- 2 Check:

$$Request_i \leq Available?$$

If no, the process waits.

- 3 Temporarily allocate:

$$Available := Available - Request_i$$

$$Allocation_i := Allocation_i + Request_i$$

$$Need_i := Need_i - Request_i$$

- 4 Run the safety algorithm.

Decision

Grant the request only if the resulting state remains safe.

Prevention vs. Avoidance

	Prevention	Avoidance
Basic idea	Break at least one necessary condition	Avoid unsafe states
Decision style	Rules imposed in advance	Dynamic allocation decision
Maximum demand	Not necessarily required	Typically required
Flexibility	More conservative	More dynamic
Main mechanism	Ordering, preemption, allocation rules	Safety algorithms
Typical algorithm	Resource ordering	Banker's Algorithm

Remember

Prevention: "How can I make deadlock impossible?"

Avoidance: "Can I safely grant this request now?"

Exam-Oriented Concept Check

- ① Why is a cycle sufficient for deadlock with single-instance resources?
- ② Why is a cycle not sufficient with multiple resource instances?
- ③ Which prevention strategy uses resource ordering?
- ④ What is the difference between a safe and an unsafe state?
- ⑤ Why is

$$Need = Max - Allocation$$

used by Banker's Algorithm?

- ⑥ What happens if a request exceeds the process's declared Need?

Summary

- Deadlock occurs when processes wait indefinitely for resources held by one another.
- Resources may be reusable or consumable.
- RAG models process-resource relationships.
- Four conditions: mutual exclusion, hold and wait, no preemption, circular wait.
- Prevention breaks at least one condition.
- Avoidance prevents the system from entering an unsafe state.
- Banker's Algorithm uses Allocation, Max, Need and Available to find a safe sequence.

Model → Analyze → Prevent / Avoid

Key Takeaways

Three rules

- ① Four conditions must coexist for deadlock.
- ② Break one condition $\Rightarrow$ prevention.
- ③ Stay in a safe state $\Rightarrow$ avoidance.

Deadlock $\rightarrow$ RAG $\rightarrow$ Conditions $\rightarrow$ Prevention $\rightarrow$ Avoidance $\rightarrow$ Banker's Algorithm